



Operations Research

Prof. Dr. Martin Bichler,
Halil Bayrak, Janik Bürgermeister, Teodora Dobos,
Jan Höhener, Abheek Ghosh

Sample solution to the demo tasks 8

Demo Exercises

Exercise D8.1. *Unimodularity*

Problem

- a) Determine for each of the following matrices whether it is totally unimodular. Justify your answer.

(i) $A = \begin{pmatrix} -1 & 0 & -1 \\ 1 & 1 & 0 \\ 1 & 1 & -1 \end{pmatrix}$

(ii) $B = \begin{pmatrix} 1 & 0 & -1 & 0 \\ 1 & -1 & 0 & 0 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & -1 \end{pmatrix}$

(iii) $C = \begin{pmatrix} -1 & 0 & 0 & -1 & 0 & 0 \\ -1 & -1 & 0 & 0 & -1 & 0 \\ 0 & -1 & -1 & 0 & 0 & -1 \end{pmatrix}$

- b) Determine all possible assignments for x and y such that D is totally unimodular. Justify your answer.

$$D = \begin{pmatrix} x & 1 & -1 \\ 0 & 1 & 0 \\ -1 & y & 1 \end{pmatrix}$$

Solution

- a) (i) A is not totally unimodular because A contains a square submatrix whose determinant does not take on a value from $\{-1, 0, 1\}$:

$$\det \begin{pmatrix} -1 & -1 \\ 1 & -1 \end{pmatrix} = 1 + 1 = 2 \notin \{-1, 0, 1\}$$

- (ii) B is totally unimodular because the following conditions are satisfied, where $M_1 = 1, 4$ and $M_2 = 2, 3$:

- $b_{ij} \in \{-1, 0, 1\}$ for all i, j .
- Each column contains at most two non-zero coefficients.
- There exists a partition (M_1, M_2) of the set M of rows, such that the following statements are fulfilled:
 - If two non-zero entries in a column of B have the same sign, then one row is in M_1 and the other in M_2 .
 - If two non-zero entries in a column of B have different signs, then both rows are either in M_1 or in M_2 .

(iii) From the lecture, it is known that if A is totally unimodular, then $-A$ and $[A, I]$ are also totally unimodular. Furthermore, according to the lecture, a matrix with consecutive ones in its columns is totally unimodular.

Now consider the matrix $-C = [Z, I]$, where Z comprises the first three columns and I comprises the last three columns of the matrix $-C$. Since Z satisfies the sufficient condition for TU with $M_1 = \{1, 3\}$ and $M_2 = \{2\}$, Z is totally unimodular. I is the identity matrix and is therefore also totally unimodular. It follows that $-C = [Z, I]$, and therefore C , is totally unimodular.

b) We consider the square submatrices of size 1×1 , 2×2 , and 3×3 one by one.

Obviously, x and y must be in $\{-1, 0, 1\}$ for D to be totally unimodular, because otherwise D would contain a 1×1 submatrix whose determinant would not lie in $\{-1, 0, 1\}$.

As soon as a 2×2 submatrix contains a 0, the corresponding determinant takes a value in $\{-1, 0, 1\}$. Therefore, it is sufficient to consider only the 2×2 submatrices that contain no zeros.

$$\det \begin{pmatrix} x & 1 \\ -1 & y \end{pmatrix} = xy + 1 \quad \text{Thus, } x \text{ and } y \text{ must not simultaneously take the values } -1 \text{ or } 1.$$

$$\det \begin{pmatrix} x & -1 \\ -1 & 1 \end{pmatrix} = x - 1 \quad \text{Thus, it must hold that } x \neq -1.$$

$$\det \begin{pmatrix} 1 & -1 \\ y & 1 \end{pmatrix} = 1 + y \quad \text{Thus, it must hold that } y \neq 1.$$

$$\det \begin{pmatrix} x & 1 & -1 \\ 0 & 1 & 0 \\ -1 & y & 1 \end{pmatrix} = x - 1 \quad \text{Thus, it must hold that } x \neq -1.$$

Based on these observations, D is totally unimodular for the following variable assignments:

- $x = y = 0$
- $x = 0, y = -1$

- $x = 1, y = 0$
- $x = 1, y = -1$

Exercise D8.2. DGS Algorithm

Problem

Consider the valuation matrix stated below. Let $p^0 = (0, 0, 0, 0)$ be the initial price vector.

	A	B	C	D
Bidder 1	9	7	4	2
Bidder 2	6	4	3	3
Bidder 3	3	1	7	2
Bidder 4	2	4	6	1

Compute an optimal allocation using the DGS algorithm and state a competitive equilibrium price vector for all bidders. At every step, state all overdemanded sets of items. If multiple occur at the same step, pick one at random to proceed to the next step.

Solution

$$\begin{array}{c}
 \begin{array}{cccc} A & B & C & D \\ \begin{pmatrix} \mathbf{9} & 7 & 4 & 2 \\ \mathbf{6} & 4 & 3 & 3 \\ 3 & 1 & \mathbf{7} & 2 \\ 2 & 4 & \mathbf{6} & 1 \end{pmatrix} & \xrightarrow{p_1=(1,0,0,0)} & \begin{array}{cccc} A & B & C & D \\ \begin{pmatrix} \mathbf{8} & 7 & 4 & 2 \\ \mathbf{5} & 4 & 3 & 3 \\ 2 & 1 & \mathbf{7} & 2 \\ 1 & 4 & \mathbf{6} & 1 \end{pmatrix} & \xrightarrow{p_2=(2,0,0,0)} & \begin{array}{cccc} A & B & C & D \\ \begin{pmatrix} \mathbf{7} & \mathbf{7} & 4 & 2 \\ \mathbf{4} & \mathbf{4} & 3 & 3 \\ 1 & 1 & \mathbf{7} & 2 \\ 0 & 4 & \mathbf{6} & 1 \end{pmatrix} \end{array} \end{array} \\
 \xrightarrow{p_3=(2,0,1,0)} \begin{array}{cccc} A & B & C & D \\ \begin{pmatrix} \mathbf{7} & \mathbf{7} & 3 & 2 \\ \mathbf{4} & \mathbf{4} & 2 & 3 \\ 1 & 1 & \mathbf{6} & 2 \\ 0 & 4 & \mathbf{5} & 1 \end{pmatrix} & \xrightarrow{p_4=(2,0,2,0)} & \begin{array}{cccc} A & B & C & D \\ \begin{pmatrix} \mathbf{7} & \mathbf{7} & 2 & 2 \\ \mathbf{4} & \mathbf{4} & 1 & 3 \\ 1 & 1 & \mathbf{5} & 2 \\ 0 & 4 & \mathbf{4} & 1 \end{pmatrix} & \xrightarrow{p_5=(3,1,3,0)} & \begin{array}{cccc} A & B & C & D \\ \begin{pmatrix} \mathbf{6} & \mathbf{6} & 1 & 2 \\ \mathbf{3} & \mathbf{3} & 0 & \mathbf{3} \\ 0 & 0 & \mathbf{4} & 2 \\ -1 & \mathbf{3} & \mathbf{3} & 1 \end{pmatrix} \end{array}
 \end{array}
 \end{array}$$

In the first round, items A and C are overdemanded. Since by definition we seek the minimal overdemanded set, we pick one at random and raise the price for it, and the same holds for the second round. Now, since we picked A , item C remains overdemanded. We increase the price for it, and in the next round it continues being overdemanded, so we raise its price twice. This time set A, B, C is overdemanded, since 4 bidders demand 3 items. Prices are raised for the set by 1. Now an optimal allocation can be seen: $\mu(1) = A, \mu(2) = D, \mu(3) = C, \mu(4) = B$ and the algorithm terminates since it arrives at a feasible allocation and overdemand is resolved.

Since the DGS implements minimal competitive equilibrium prices, a valid price vector is $p_5 = (3, 1, 3, 0)$

Exercise D8.3. *Matroids*

Problem

First, we define the concept of an independence system. Compare this particularly with the definition of a matroid from the lecture.

Let E be a finite set and $\mathcal{I}$ be a non-empty collection of subsets of E , i.e., $\mathcal{I} \subset \mathcal{P}(E)$. Then $(E, \mathcal{I})$ is an **independence system** if and only if the following hold:

- 1) $\emptyset \in \mathcal{I}$,
 - 2) if $B \in \mathcal{I}$ and $A \subset B$, then $A \in \mathcal{I}$.
- a) Given the sets $E_1 = \{1, 2, 3, 4\}$ and $\mathcal{I}_1 = \{\emptyset, \{1\}, \{2\}, \{3\}, \{4\}, \{1, 2\}, \{1, 3\}, \{1, 4\}, \{2, 3\}, \{1, 2, 3\}, \{1, 2, 4\}\}$. Prove or disprove: $U_1 = (E_1, \mathcal{I}_1)$ is an independence system.
- b) Given the sets $E_2 = \{1, 2, 3, 4\}$ and $\mathcal{I}_2 = \{\emptyset, \{1\}, \{2\}, \{3\}, \{4\}, \{1, 2\}, \{1, 3\}, \{1, 4\}, \{2, 3\}, \{2, 4\}, \{3, 4\}, \{1, 2, 4\}, \{2, 3, 4\}\}$. Prove or disprove: $U_2 = (E_2, \mathcal{I}_2)$ is a matroid.
- c) Let E_3 be a finite set and let $\mathcal{I}_3 = \{S \subseteq E_3 : |S| \leq k\}$ with $k \in \mathbb{N}$. Show that $U_3 = (E_3, \mathcal{I}_3)$ is a matroid.

Solution

- a) U_1 is not an independence system, as $\{2, 4\} \subseteq \{1, 2, 4\} \in \mathcal{I}_1$, but $\{2, 4\} \notin \mathcal{I}_1$.
- b) Let $A = \{1, 3\} \in \mathcal{I}_2$ and $B = \{1, 2, 4\} \in \mathcal{I}_2$. The independence system U_2 is not a matroid, because exists no element e in $B \setminus A$, such that $A \cup \{e\} \in \mathcal{I}_2$.
- c) Clearly, we have $\emptyset \in \mathcal{I}_3$, as $|\emptyset| = 0 \leq k \in \mathbb{N}$. Let $A \in \mathcal{I}_3$ and $B \subseteq A$. Then $B \in \mathcal{I}_3$, due to $|B| \leq |A| \leq k$. Therefore, U_3 is an independence system. To show U_3 is a matroid, consider $A, B \in \mathcal{I}_3$ with $|B| < |A| \leq k$. Choose an arbitrary element $e \in A \setminus B$. Then $|B \cup \{e\}| \leq |A| \leq k$ by $B \cup \{e\} \in \mathcal{I}_3$.

Exercise D8.4. *Tips and tricks for network modeling*

Problem

A generalization of the Maximum Flow Problem (MFP) is the so-called Minimum Cost Network Flow Problem (MCNFP), which is defined as follows. Given a network $N = (V, E, c, u, b)$ with nodes V , edges E , transportation costs (per unit flow) $c : E \rightarrow \mathbb{R}$, capacities $u : E \rightarrow \mathbb{R}^+$, and demand $b : V \rightarrow \mathbb{R}$, determine a cost-minimal flow $f : E \rightarrow \mathbb{R}$ such that all demands are met, i.e., $\sum_i f(i, j) + b_j = \sum_i f(j, i)$ for all $j \in V$. A positive demand corresponds to a production, and a negative demand corresponds to a consumption.

- a) Show that you can convert any MaxFlow Problem (MFP) into a Minimum Cost Network Flow Problem (MCNFP). To do this, suitably extend the network of the MFP with transportation costs so that an (optimal) solution of the MCNFP corresponds one-to-one to an (optimal) solution of the MFP.

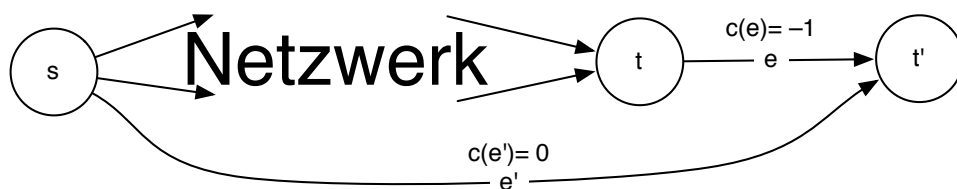
Remark: This procedure is called the *reduction* of MFP to MCNFP. You have shown

that MCNFP is at least as difficult as MFP because you can solve any MFP with an MCNFP algorithm and a small additional modeling effort.

- b) Given is a network flow problem with network N_1 where there are more than two nodes with a non-zero demand. Provide a transformed network N_2 that has only one source s and one sink t and whose cost-optimal flows correspond to the cost-optimal flows in the original network up to transformation. (Thus, a reduction from 'MCNFP with multiple sources and sinks' to 'MCNFP with only one source and one sink' is sought.)
- c) Transportation costs can also be negative. On such edges, negative costs result in a profit, for example, in the form of value creation. Reduce MCNFP to 'MCNFP without negative transportation costs'. It is assumed that an edge with negative costs has a finite capacity.
- d) Given is a network-like model (nodes, edges, capacities) with additional capacity constraints for the flow passing through a *node*. Transform this into an equivalent network where capacities are defined only on edges.

Solution

- a) We add a new node t' and an edge $e = (t, t')$ with $c(e) = -1$. We create a large demand at t' , i.e., $b(s) = -b(t') = |E| \cdot \max_{e \in E}(u(e))$, and adjust the edge from t' accordingly: $u(e) = -b(t')$. To ensure that the demand can be met (we do not know the maximum possible flow value, which we want to determine), we also add a cost-free edge e' from s to t' : $u(e') = -b(t')$, $c(e') = 0$. For all other edges from the original network, the costs are now zero. In an optimal MCNFP solution, all flow that does not run over the two new edges corresponds to a solution of the Max Flow problem.



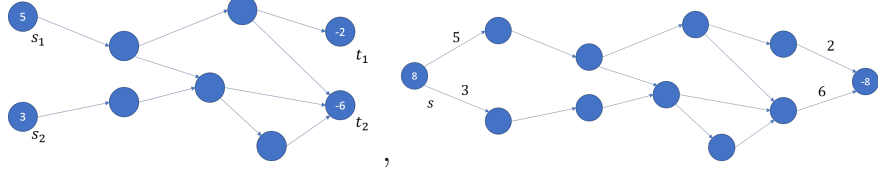
Remark: It is useful to understand why one cannot simply assign a cost of -1 to all edges. The reason is the different path lengths. In such a case, it could happen that it is more cost-effective to route less flow over longer paths.

- b) We add a new source node that provides the aggregated production of the original source nodes, as well as a new sink node that has the aggregated demand of the original sinks. Additionally, the capacities of the new edges are adjusted accordingly so that the same amount of flow can enter and leave an original source or sink as before. The new nodes are suitably placed before the original sources and after the original sinks.

The construction is as follows. Let $s_1, \dots, s_n$ be the nodes with positive demand (i.e., the sources with $b(s_i) > 0$ for $1 \leq i \leq n$) and $t_1, \dots, t_m$ be the nodes with negative demand (i.e., the sinks with $b(t_j) < 0$ for $1 \leq j \leq m$). Additionally, let s and t be

artificial nodes, such that there exists an edge from s to s_i with $u(s, s_i) = b(s_i)$ and an edge from t_j to t with $u(t_j, t) = |b(t_j)|$ for each $1 \leq i \leq n, 1 \leq j \leq m$. Furthermore, $b(s) = \sum_{i=1}^n b(s_i)$ and $b(t) = \sum_{j=1}^m b(t_j)$.

Here is an example for two sources.

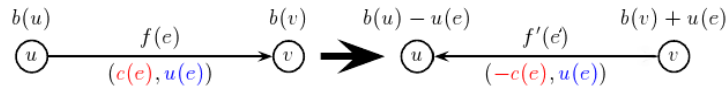


- c) Let $e = (v_1, v_2)$ be an edge with negative cost $c(e)$ and capacity $u(e)$ in the network N . We transform N to N' as follows. First, the edge between nodes v_1 and v_2 in N' now points in the opposite direction, that is, from v_2 to v_1 , or more precisely, $e' = (v_2, v_1) \in E'$ with cost $c(e') = -c(e) > 0$. The capacity of e' is equal to that of e , i.e., $u'(e') = u(e)$. Additionally, we adjust the demands of nodes v_1 and v_2 :

$$b'(v_1) = b(v_1) - u(e) \text{ und } b'(v_2) = b(v_2) + u(e).$$

The demands of the nodes have now been adjusted as if a flow of $u(e)$ has already passed through (v_1, v_2) . However, if it is more cost-effective in the network N not to use the full capacity of e , this will result in a flow $f'(e')$ at positive costs $f'(e') \cdot c'(e')$ from v_2 to v_1 in N' . The new flow value is thus intuitively the shortfall of the maximum possible flow value in the original network.

$$f(e) = u(e) - f'(e')$$



- d) Each such node is replaced by two new nodes and an edge between them. One node receives all incoming connections, the other receives all outgoing edges, and the edge between them has no costs but the capacity of the original node.

